Geometry II

Rotating calipers, half-plane intersection, closest pair, angular sweep

Learning goals

- Enumerate **antipodal pairs** with rotating calipers in O(n): diameter, width, bounding boxes
- Intersect n half-planes in O(n log n) and use it inside a binary search
- Prove the O(n log n) bound for **closest pair** and implement it with a sweep
- Sort by angle **exactly** and run two-pointer / event sweeps around a point
- Know when to stay in integers and when `long double` + ε is unavoidable

Prereqs: Year week 31 (cross product, hull), Regional 08 (precision, intersections)

Motivating problems

1. Farthest pair among $2 \cdot 10^5$ points (POJ 2187 *Beauty Contest*, Kattis *roberthood*)
2. Point inside a convex polygon farthest from its boundary (POJ 3525 *Most Distant Point from the Sea*)
3. Closest pair among $2 \cdot 10^5$ points (CSES 2194 *Minimum Euclidean Distance*)
4. Count triangles / quadrilaterals around a point (CF 1284E *New Year and Castle Construction*)

All four have $O(n^2)$ brute forces and $O(n \log n)$ intended solutions.

Toolkit recap (integers first)

```
typedef long long ll;
struct P {
    ll x, y;
    P operator-(const P& o) const { return {x - o.x, y - o.y}; }
    bool operator<(const P& o) const { return tie(x, y) < tie(o.x, o.y); }
};
ll cross(P a, P b) { return a.x * b.y - a.y * b.x; }
ll cross(P o, P a, P b) { return cross(a - o, b - o); }   // > 0: left turn
ll dist2(P a, P b) { P d = a - b; return d.x * d.x + d.y * d.y; }
```

Coordinates $\le 10^9$ $\Rightarrow$ cross products $\le 8 \cdot 10^{18}$ — fits `long long` **barely**.

Coordinates $\le 10^9$ with products of crosses $\Rightarrow$ `__int128`.

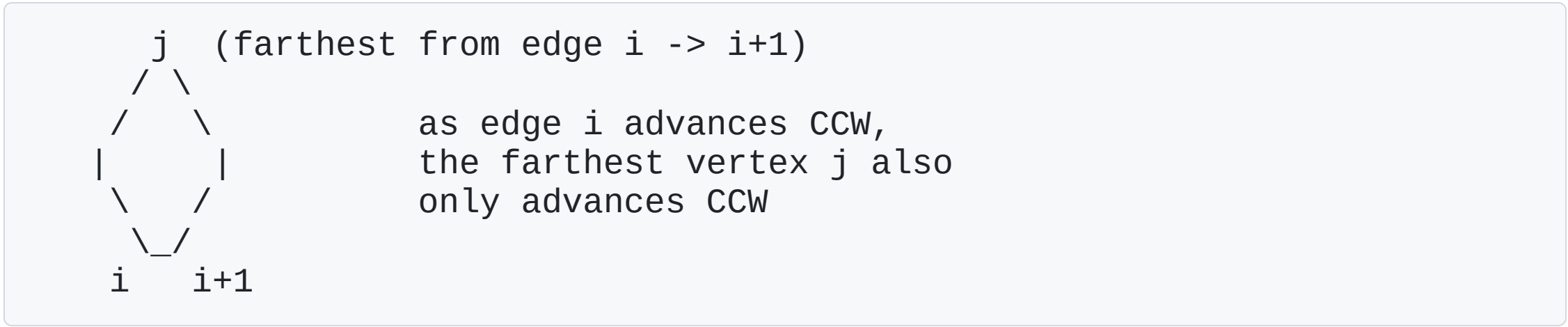
Convex hull (monotone chain) — recap

```
vector<P> convexHull(vector<P> pts) {          // CCW, collinear points removed
    sort(pts.begin(), pts.end());
    pts.erase(unique(pts.begin(), pts.end()), pts.end());
    int n = pts.size();
    if (n < 3) return pts;
    vector<P> h(2 * n); int k = 0;
    for (int i = 0; i < n; i++) {
        while (k >= 2 && cross(h[k-2], h[k-1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    for (int i = n - 2, t = k + 1; i >= 0; i--) {
        while (k >= t && cross(h[k-2], h[k-1], pts[i]) <= 0) k--;
        h[k++] = pts[i];
    }
    h.resize(k - 1);
    return h;
}
```

Every technique today starts from a clean CCW hull **without collinear points**.

Rotating calipers: the idea

Two parallel support lines touching the polygon on opposite sides, rotated together.



- **Antipodal pair**: vertices admitting parallel support lines
- There are O(n) of them; the farthest pair is one of them
- j is monotone in i ⇒ two pointers, O(n) total

Diameter with calipers

```
ll diameter2(const vector<P>& h) {      // CCW hull, no collinear points
    int n = h.size();
    if (n == 1) return 0;
    if (n == 2) return dist2(h[0], h[1]);
    ll best = 0;
    for (int i = 0, j = 1; i < n; i++) {
        int ni = (i + 1) % n;
        while (cross(h[ni] - h[i], h[(j + 1) % n] - h[j]) > 0) j = (j + 1) % n;
        best = max({best, dist2(h[i], h[j]), dist2(h[ni], h[j])});
    }
    return best;
}
```

Advance j while the area of triangle (i, i+1, j) increases, i.e. while the next edge at j still "turns toward" edge i. Same loop gives the **width**: min over i of `cross(e_i, h[j] − h[i]) / |e_i|`.

What else the calipers give

Quantity	Calipers variant
Diameter	1 caliper pair, antipodal vertices
Width	edge–vertex antipodal pairs
Min-area / min-perimeter bounding rectangle	4 calipers (one edge flush)
Max distance between two convex polygons	calipers on both hulls
Min distance between disjoint convex polygons	calipers, or Minkowski difference
Largest triangle with vertices on hull	2 pointers per fixed vertex ($O(n^2)$)

The "one side is flush with a hull edge" lemma (Freeman–Shapira) is why the

Half-plane intersection (HPI)

Half-plane = points to the **left** of directed line `p → p + d`.
The intersection of n half-planes is a convex polygon (possibly empty/unbounded).

Algorithm (O(n log n)):

1. Add a huge bounding box (4 half-planes) ⇒ always bounded
2. Sort by angle of `d`
3. Deque: for each new half-plane, pop from the **back** while the last vertex is outside it; pop from the **front** likewise; handle parallels
4. Finally trim back against front and front against back
5. Vertices = intersections of consecutive deque lines

HPI code (core loop)

```
sort(H.begin(), H.end(), [](const HP& a, const HP& b) { return a.ang < b.ang; });
deque<HP> dq;
for (auto& h : H) {
    while (dq.size() > 1 && h.out(inter(dq[dq.size()-1], dq[dq.size()-2]))) dq.pop_back();
    while (dq.size() > 1 && h.out(inter(dq[0], dq[1]))) dq.pop_front();
    if (!dq.empty() && fabsl(cross(h.d, dq.back().d)) <= EPS) {
        if (dot(h.d, dq.back().d) < 0) return {};      // opposite parallel: empty
        if (h.out(dq.back().p)) dq.pop_back();         // same direction: keep tighter
        else continue;
    }
    dq.push_back(h);
}
while (dq.size() > 2 && dq[0].out(inter(dq[dq.size()-1], dq[dq.size()-2]))) dq.pop_back();
while (dq.size() > 2 && dq.back().out(inter(dq[0], dq[1]))) dq.pop_front();
if (dq.size() < 3) return {};
```

`out(r)`: `cross(d, r − p) < −EPS`. Use `long double`.

HPI applications

- **Polygon kernel**: points that see the whole polygon = ∩ of edge half-planes (POJ 3335 *Rotating Scoreboard*, POJ 1279 *Art Gallery*)
- **Largest inscribed circle** of a convex polygon: binary search r, shift every edge inward by r, check non-empty (POJ 3525)
- **Feasibility of 2-variable linear constraints**: `a x + b y ≤ c` for many rows
- **Voronoi cell** of one site: ∩ of perpendicular-bisector half-planes, O(n log n)
- LP in 2D: optimize over the HPI polygon (or Seidel's randomized LP, expected O(n))

Closest pair: sweep line

Sort by x. Sweep left to right keeping an **active set ordered by y** of points with x ≥ current x − d, where d = best distance so far.

For the new point, only check active points with $|y - y_o| \le d$.

```
      d     d
   |<------|------>|      the d × 2d box left of the new point
   |  . .  |       |      holds O(1) points: they are pairwise ≥ d apart
   |   .   * new   |      (packing: at most 6–8 such points)
   |  . .  |       |
```

Each point inserted/erased once, O(1) checks per point ⇒ **O(n log n)**.

Closest pair code

```
ll closestPair(vector<pair<ll, ll>> pts) {          // returns squared distance
    sort(pts.begin(), pts.end());
    multiset<pair<ll, ll>> active;                  // (y, x)
    ll best = LLONG_MAX; size_t left = 0;
    for (auto& [x, y] : pts) {
        ll d = (ll)ceil(sqrt((long double)best));
        while (left < pts.size() && pts[left].first < x - d) {
            active.erase(active.find({pts[left].second, pts[left].first}));
            left++;
        }
        for (auto it = active.lower_bound({y - d, LLONG_MIN});
             it != active.end() && it->first <= y + d; ++it) {
            ll dx = x - it->second, dy = y - it->first;
            best = min(best, dx * dx + dy * dy);
        }
        active.insert({y, x});
    }
    return best;
}
```

Closest pair: other routes

- **Divide & conquer** (Shamos–Hoey): split by median x, merge by y, check the strip;
  $T(n) = 2T(n/2) + O(n)$ with merge sort ⇒ $O(n \log n)$
- **Randomized grid** (Rabin / Khuller–Matias): expected $O(n)$ with hashing
- Variant: closest pair between **two colors** — same sweep, only compare across colors
- Variant: k-th closest / all pairs within r — grid buckets of size r

Angular sweep: exact angle sort

```
int half(P a) { return (a.y < 0 || (a.y == 0 && a.x < 0)) ? 1 : 0; }
bool angleLess(P a, P b) {
    if (half(a) != half(b)) return half(a) < half(b);
    return cross(a, b) > 0;
}
```

- Splits the plane into [0, π) and [π, 2π); inside a half, cross product decides
- No `atan2` ⇒ no precision ties for integer input
- Collinear-with-origin points compare as equal — decide what the problem wants

Angular sweep pattern: two pointers around a center

Count triangles of points strictly containing the origin (no 3 collinear with origin):

- Total C(n, 3) minus triangles lying in an open half-plane through 0
- Each "bad" triangle has a unique **first** vertex i (CCW) with the other two in the half-turn (θ_i, $\theta_i + \pi$)
- Two pointers on the sorted, conceptually doubled, array: bad += C(k_i, 2)

```
for (int i = 0, j = 0; i < n; i++) {
    if (j < i + 1) j = i + 1;
    while (j < i + n && cross(pts[i], pts[j % n]) > 0) j++;
    ll k = j - i - 1;
    bad += k * (k - 1) / 2;
}
```

CF 1284E: same count with 4 points per center; run it with every point as center.

Angular sweep pattern: rotating events

Max points covered by a circle of radius r (POJ 1981):

- Optimal circle can be assumed to have a point on its boundary
- Fix that point p; each other point q within 2r gives an **arc of center angles** `[φ − α, φ + α]` with φ = angle(q − p), α = acos(|pq| / 2r)
- Sweep the angle events (+1 at start, −1 at end, starts before ends on ties)
- $O(n^2 \log n)$ total

Same template: visibility from a point, "rotate a ray and count crossings", maximum points on one side of a line through a fixed point.

Precision policy

Situation	Choice
Input integers, predicates only (hull, orientation, angle sort)	`long long` / `__int128`, **no ε**
Distances compared	compare **squared** distances in integers
Intersections, HPI, circles	`long double`, ε ≈ 1e−9 relative to scale
Output real numbers	compute in `long double`, print with `setprecision(9)`

Rule: introduce floating point as **late** as possible, and ε only where a floating value is compared.

Complexity summary

Algorithm	Time
Convex hull	$O(n \log n)$
Calipers (diameter, width, rectangle)	$O(n)$ after hull
Half-plane intersection	$O(n \log n)$
Closest pair (sweep or D&C)	$O(n \log n)$
Angular sort + two pointers per center	$O(n \log n)$ per center
Circle cover via events	$O(n^2 \log n)$

Common pitfalls

- Calipers on a hull **with collinear points** or duplicate points → infinite loop / wrong j
- n = 1, 2 hulls; all points collinear (hull has 2 vertices)
- HPI: forgetting the bounding box; wrong side convention; `double` instead of `long double`
- HPI degenerate result (point/segment) — decide whether it counts as non-empty
- Closest pair with `int` squares, or window bound rounded down
- `atan2` ties for parallel vectors — use the exact comparator
- Angular two-pointer: forgetting the wrap-around (`j % n`, `j < i + n`)

Practice problems

- CSES 2195 *Convex Hull*, CSES 2194 *Minimum Euclidean Distance*
- POJ 2187 *Beauty Contest*, Kattis *roberthood*, UVa 10173 *Smallest Bounding Rectangle*
- POJ 3335 *Rotating Scoreboard*, POJ 1279 *Art Gallery*, POJ 3525 *Most Distant Point from the Sea*
- Kattis *closestpair2*, Library Checker *Sort Points by Argument*
- Codeforces 598C *Nearest vectors*, 1284E *New Year and Castle Construction*
- POJ 1981 *Circle and Points*, Codeforces 1142C *U2*
- Stretch: Codeforces 1025F *Disjoint Triangles*

Summary

- Convex hull first; then **calipers** turn "farthest/thinnest" questions into O(n)
- **HPI** = sort by angle + deque; pairs well with binary search on the answer
- **Closest pair**: sweep with a y-ordered window; packing argument ⇒ O(1) checks
- **Angular sweep**: exact angle sort + two pointers or events around a center
- Next lecture: bridges, biconnectivity, 2-SAT, Euler tours